

SCR2043 OPERATING SYSTEMS

Name : YASMIN BATRISYIA BINTI
ZAHIRUDDIN

Student ID : A23CS0201

Section : 02

Marks

This lab assessment is designed to test your understanding and skills on some basic concepts and tools related to process monitoring and management in operating system. Please follow the instructions carefully and submit your answers in this word document and rename the file as **os-lab-assessment02-studentname-matricno.docx**.

Essential Steps Before Starting Lab Assessment 2:

1. Download necessary source codes:

Use the `wget` command to retrieve the following source code files to your Linux (or WSL or MacOS) environment:

```
wget -O mainprocess.c https://rebrand.ly/mainprocess_c
wget -O subprocess1.c https://rebrand.ly/subprocess1_c
wget -O subprocess2.c https://rebrand.ly/subprocess2_c
```

2. Compile the source files:

Use the `gcc` compiler to create executable files from the source code.

```
gcc mainprocess.c -o mainprocess
gcc subprocess1.c -o subprocess1
gcc subprocess2.c -o subprocess2
```

3. Execute the dummy processes:

Run all the dummy processes

```
./mainprocess &
```

Press **enter** two times.

4. The dummy processes are running for 2 hours. If you took longer than 2 hours on questions 1-9, please restart the main process with `./mainprocess &`.

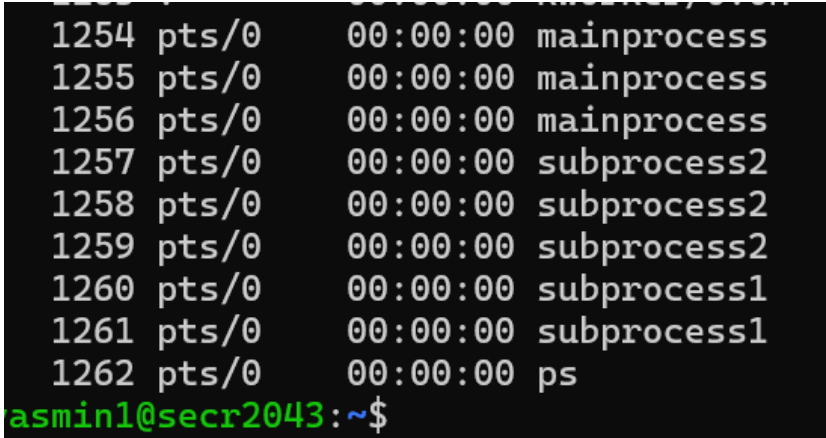
Lab Assessment 2 : Linux Process Monitoring and Management

Instructions:

1. Carefully execute each command as instructed in the questions.
2. Write down the exact command used for each task.
3. Capture a screenshot of the command's output.

Question 1

Use the `ps` command with the appropriate option to display a complete list of all running processes within the Linux operating system.

Command
<code>ps -ef</code>
Output


Question 2

Employ the `ps` command with necessary options to unveil comprehensive details about each running process.

Command									
ps -eo pid,ppid,user,stat,start,time,cmd									
Output									
892	857	yasmin1	S	13:07:08	00:00:00	./mainprocess			
893	892	yasmin1	S	13:07:08	00:00:00	./mainprocess			
894	892	yasmin1	S	13:07:08	00:00:00	./mainprocess			
895	894	yasmin1	S	13:07:08	00:00:00	./subprocess2			
896	893	yasmin1	S	13:07:08	00:00:00	./subprocess1			
897	893	yasmin1	S	13:07:08	00:00:00	./subprocess1			
898	894	yasmin1	S	13:07:08	00:00:00	./subprocess2			
899	894	yasmin1	S	13:07:08	00:00:00	./subprocess2			
900	2	root	I	13:07:24	00:00:02	[kworker/0:0-events]			
901	2	root	I	13:07:45	00:00:00	[kworker/u5:3-events_unbound]			
903	857	yasmin1	R+	13:09:39	00:00:00	ps -eo pid,ppid,user,stat,start,time,cmd			

Question 3

Use the `ps` command with some tools to only list processes named "subprocess" and show some info about them.

Command
<pre>ps -ef grep subprocess grep -v grep</pre>
Output
<pre>yasmin1@secre2043:~\$ ps -ef grep subprocess grep -v grep yasmin1 895 894 0 13:07 pts/0 00:00:00 ./subprocess2 yasmin1 896 893 0 13:07 pts/0 00:00:00 ./subprocess1 yasmin1 897 893 0 13:07 pts/0 00:00:00 ./subprocess1 yasmin1 898 894 0 13:07 pts/0 00:00:00 ./subprocess2 yasmin1 899 894 0 13:07 pts/0 00:00:00 ./subprocess2</pre>

Question 4

Execute the `ps` command, specifying options that reveal only the following columns:

- Process ID (`pid`)
- Owner of the process (`user`)
- CPU percentage (`pcpu`)
- Memory percentage (`pmem`)
- Command (`cmd`)

Command				
ps -eo pid,user,pcpu,pmem,cmd				
Output				
<pre>892 yasmin1 0.0 0.1 ./mainprocess 893 yasmin1 0.0 0.0 ./mainprocess 894 yasmin1 0.0 0.0 ./mainprocess 895 yasmin1 0.0 0.1 ./subprocess2 896 yasmin1 0.0 0.1 ./subprocess1 897 yasmin1 0.0 0.1 ./subprocess1 898 yasmin1 0.0 0.1 ./subprocess2 899 yasmin1 0.0 0.1 ./subprocess2 900 root 0.7 0.0 [kworker/0:0-events] 901 root 0.0 0.0 [kworker/u5:3-events_unbound] 904 root 0.0 0.0 [kworker/u6:2-flush-8:0] 912 root 0.4 0.0 [kworker/1:0-ata_sff] 913 yasmin1 150 0.4 ps -eo pid,user,pcpu,pmem,cmd</pre>				

Question 5

Building on the ps command used in Question 4, can you add an option to sort the listed processes by their memory usage (pmem)?

Command	
ps -eo pid,user,pcpu,pmem,cmd --sort=-pmem	
Output	
<pre> yasmin1@seer2043:~\$ ps -eo pid,user,pcpu,pmem,cmd --sort=-pmem PID USER %CPU %MEM CMD 364 root 0.0 2.7 /sbin/multipathd -d -s 689 root 0.0 2.3 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal 662 root 0.0 1.3 /usr/libexec/udisks2/udisksd 1 root 0.4 1.3 /sbin/init 714 root 0.0 1.3 /usr/sbin/ModemManager 533 systemd+ 0.0 1.2 /usr/lib/systemd/systemd-resolved 307 root 0.0 1.2 /usr/lib/systemd/systemd-journald 525 systemd+ 0.0 0.9 /usr/lib/systemd/systemd-networkd 660 root 0.0 0.8 /usr/lib/systemd/systemd-logind 654 polkitd 0.0 0.8 /usr/lib/polkit-1/polkitd --no-debug 852 root 0.0 0.8 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups 536 systemd+ 0.0 0.7 /usr/lib/systemd/systemd-timesyncd 373 root 0.0 0.7 /usr/lib/systemd/systemd-udev 856 yasmin1 0.2 0.7 sshd: yasmin1@pts/0 683 syslog 0.0 0.5 /usr/sbin/rsyslogd -n -iNONE 854 root 0.0 0.5 sshd: yasmin1 [priv] 650 message+ 0.0 0.5 @dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-o 857 yasmin1 0.0 0.4 -bash 915 yasmin1 150 0.4 ps -eo pid,user,pcpu,pmem,cmd --sort=-pmem 754 root 0.0 0.2 /usr/sbin/cron -f -P 775 root 0.0 0.1 /sbin/agetty -o -p -- \u --noclear - linux 892 yasmin1 0.0 0.1 ./mainprocess 895 yasmin1 0.0 0.1 ./subprocess2 896 yasmin1 0.0 0.1 ./subprocess1 897 yasmin1 0.0 0.1 ./subprocess1 898 yasmin1 0.0 0.1 ./subprocess2 899 yasmin1 0.0 0.1 ./subprocess2 893 yasmin1 0.0 0.0 ./mainprocess 894 yasmin1 0.0 0.0 ./mainprocess </pre>	

Question 6

Construct a command using ps, suitable options, and any additional tools to visualize the hierarchical structure (tree-like) of the following processes:

- "mainprocess"
- "subprocess1"
- "subprocess2"

Command	
ps -ejH grep -E "mainprocess subprocess1 subprocess2"	
Output	
<pre> yasmin1@seer2043:~\$ ps -ejH grep -E "mainprocess subprocess1 subprocess2" 892 892 857 pts/0 00:00:00 mainprocess 893 892 857 pts/0 00:00:00 mainprocess 896 892 857 pts/0 00:00:00 subprocess1 897 892 857 pts/0 00:00:00 subprocess1 894 892 857 pts/0 00:00:00 mainprocess 895 892 857 pts/0 00:00:00 subprocess2 898 892 857 pts/0 00:00:00 subprocess2 899 892 857 pts/0 00:00:00 subprocess2 </pre>	

Question 7

Use `pstree` command with option that show the number of threads to each process.

Command
<code>pstree -pT</code>
Output
<pre>yasmin1@secr2043:~\$ pstree -p -t systemd(1)---ModemManager(714)---{gdbus}(724) ---{gmain}(719) ---{pool-spawner}(721) ---agetty(775) ---cron(754) ---dbus-daemon(650) ---multipathd(364)---{multipathd}(379) ---{multipathd}(380) ---{multipathd}(381) ---{multipathd}(382) ---{multipathd}(383) ---{multipathd}(384) ---polkitd(654)---{gdbus}(703) ---{gmain}(700) ---{pool-spawner}(701) ---rsyslogd(683)---{in:imklog}(705) ---{in:imuxsock}(704) ---{rs:main Q:Reg}(708) ---sshd(852)---sshd(854)---sshd(856)---bash(857)---mainprocess(892)---mainprocess(893)---subprocess1(896) ---subprocess1(897) ---mainprocess(894)---subprocess2(895) ---subprocess2(898) ---subprocess2(899) ---pstree(928) ---systemd-journal(307) ---systemd-logind(660) ---systemd-network(525) ---systemd-resolve(533) ---systemd-timesyn(536)---{sd-resolve}(588) ---systemd-udev(373) ---udisksd(662)---{cleanup}(725) ---{gdbus}(695) ---{gmain}(691) ---{pool-spawner}(692) ---{probing-thread}(717) ---unattended-upgr(689)---{gmain}(761)</pre>

Question 8

Use `renice` command to change priority level of one of process “subprocess1”.

Command
<code>ps -o pid,nice,comm -C subprocess1</code> <code>sudo renice 10 896</code>
Output
<pre>yasmin1@secr2043:~\$ ps -o pid,nice,comm -C subprocess1 PID NI COMMAND 896 0 subprocess1 897 0 subprocess1 yasmin1@secr2043:~\$ sudo renice 10 896 896 (process ID) old priority 0, new priority 10</pre>

Question 9

Terminate all running processes with the name “mainprocess”.

Command
<code>pkill mainprocess</code>
Output
<pre>yasmin1@secr2043:~\$ pkill -f mainprocess Main process (ID: 892) received signal: 15. Terminating... Main process (ID: 894) received signal: 15. Terminating... [1]+ Done ./mainprocess Main process (ID: 893) received signal: 15. Terminating...</pre>

Question 10

Write a short C or Python code (choose only one language) demonstrating multiprocessing with `fork()` and `wait()`. Compile and/or run the code. Show the output.

Source Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork(); // Create a child process

    if (pid == 0) {
        // This block runs in the child process
        printf("Child process: PID = %d\n", getpid());
    } else {
        // This block runs in the parent process
        wait(NULL); // Wait for the child to finish
        printf("Parent process: PID = %d, Child finished.\n", getpid());
    }
    return 0;
}
```

Output:

```
yasmin1@secr2043:~$ nano q10.c
yasmin1@secr2043:~$ gcc q10.c -o q10
yasmin1@secr2043:~$ ./q10
Child process: PID = 984
Parent process: PID = 983, Child finished.
```